

“Año de la Esperanza y el Fortalecimiento de la Democracia”
UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
Universidad del Perú. Decana de América



FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA

Proyecto Final: Avance 1

LABORATORIO

L14

GRUPO

7

DOCENTE

Ing. Carlos Robles Castro

INTEGRANTES

Meza Alejo, Yober Ronal	23190182
Quiroz Loli Rubby Alina	23190386
Rios Mallqui Yamila Hefzibá	23190387
Vera Vilca Edward Fabiano	23190213

Lima, Perú
2026

Primer avance: Programa de citas

Rios Mallqui Yamila Hefzibá, Quiroz Loli Rubby Alina, Meza Alejo Yober Ronal, Vera Vilca

Edward Fabiano

Facultad de Ingeniería Electrónica y eléctrica, Universidad Nacional Mayor de San Marcos

Lima, Perú

yamila.rios@unmsm.edu.pe

rubby.guiroz@unmsm.edu.pe

yober.meza@unmsm.edu.pe

edward.vera@unmsm.edu.pe

I. OBJETIVOS

- Objetivo general:
 - Desarrollar una aplicación web para la gestión de citas médicas entre pacientes y doctores.
- Objetivos específicos:
 - Implementar autenticación de usuarios.
 - Diferenciar perfiles de paciente y doctor.
 - Permitir que el paciente agende y cancele citas.
 - Permitir que el doctor visualice, confirme y cancele citas.
 - Preparar la estructura para una futura conexión con base de datos.

II. TECNOLOGÍAS UTILIZADAS

Java 21
Spring Boot
Spring MVC
Spring Security
JSP
JSTL
Bootstrap 5
Maven
Tomcat embebido

III. ESTRUCTURA DEL PROYECTO

En esta sección se describe cómo está organizado el proyecto a nivel de carpetas y archivos. La estructura sigue una organización basada en Spring MVC, separando la configuración, los controladores, los modelos, los servicios, las vistas JSP y los recursos estáticos.

Estructura:

```
citás_java/  
├── pom.xml  
├── README.md  
├── src/  
│   ├── main/  
│   └── java/  
└──
```

```
├── com/  
│   └── clinica/  
│       ├── ClinicaApplication.java  
│       ├── ServletInitializer.java  
│       ├── config/  
│       ├── controller/  
│       ├── model/  
│       ├── repository/  
│       ├── service/  
│       └── impl/  
├── resources/  
│   └── application.properties  
├── webapp/  
│   ├── resources/  
│   │   └── css/  
│   ├── WEB-INF/  
│   │   └── jsp/  
│   │       ├── login.jsp  
│   │       ├── error.jsp  
│   │       ├── acceso-denegado.jsp  
│   │       ├── paciente/  
│   │       └── doctor/  
└──
```

4.1. Archivo “pom.xml”

Este archivo pertenece a la configuración general del proyecto Maven. En este archivo se definen las dependencias necesarias para que la aplicación funcione.

Entre las principales dependencias se encuentran:

- Spring Boot Web.
- Spring Security.
- Soporte para JSP.
- JSTL.
- Tomcat embebido.

Este archivo no pertenece directamente al frontend ni al backend funcional, sino a la configuración del proyecto. Gracias a este archivo, Maven puede descargar las librerías necesarias y construir la aplicación.

4.2. Carpeta “src/main/java/com/clinica”

Esta carpeta contiene el código fuente principal desarrollado en Java. Aquí se encuentra la parte lógica del sistema, es decir, el backend de la aplicación.

Dentro de esta carpeta están las clases encargadas de:

- Iniciar la aplicación.
- Configurar Spring MVC y Spring Security.
- Controlar las rutas web.
- Representar los datos del sistema.

- Aplicar la lógica de negocio.
- Preparar la futura conexión con base de datos.

En términos generales, esta carpeta representa el núcleo del sistema.

4.3. Clase “ClinicaApplication.java ”

Esta clase pertenece al backend y es el punto de entrada principal de la aplicación.

Su función es iniciar Spring Boot y levantar el servidor local donde se ejecutará el sistema. Cuando se ejecuta el proyecto, esta clase permite que la aplicación quede disponible en una dirección como: <http://localhost:8080>

Desde ahí el usuario puede acceder al login mediante: <http://localhost:8080/signin>

4.4. Clase “ServletInitializer.java ”

Esta clase también pertenece a la configuración del backend. Su función es permitir que el proyecto pueda empaquetarse como archivo WAR. Esto es útil si en el futuro se desea desplegar la aplicación en un servidor externo compatible con Java, como **Apache Tomcat**.

En este primer avance, la aplicación puede ejecutarse localmente usando el servidor embebido de Spring Boot.

4.5. Carpeta “src/main/java/com/clinica/config ”

Esta carpeta pertenece a la configuración general del backend. Contiene clases que definen cómo se comporta la aplicación a nivel de seguridad, vistas y contraseñas. Dentro de esta carpeta se encuentran:

- **SecurityConfig.java:** Esta clase configura Spring Security, se encarga de:
 - Definir la página de login.
 - Validar usuarios.
 - Asignar roles.
 - Proteger rutas.
 - Redirigir al usuario según su perfil.
 - Controlar el cierre de sesión.
 - Mostrar acceso denegado cuando corresponde.
 - Pertenecer al backend, específicamente a la capa de seguridad.
- **WebConfig.java:** Esta clase configura algunas rutas simples que devuelven vistas JSP directamente, como: [/signin](#) [/error](#) [/acceso-denegado](#). Pertenecer a la capa de configuración MVC.
- **PasswordConfig.java:** Esta clase define el codificador de contraseñas usando BCrypt. Su función es permitir que las contraseñas no se manejen como texto plano, sino encriptadas. Pertenecer a la configuración de seguridad del backend.

4.6. Carpeta “src/main/java/com/clinica/controller ”

Esta carpeta pertenece al backend, pero actúa como puente entre el frontend y la lógica del sistema.

Los controladores reciben las peticiones del navegador y deciden qué respuesta devolver. Contiene:

- **PacienteController.java:** Controla las funcionalidades del paciente. Maneja rutas como:
 - [/paciente/dashboard](#)
 - [/paciente/agendar](#)
 - [/paciente/mis-citas](#)
 - [/paciente/cancelar-cita](#)

Desde este controlador, el paciente puede:

- Ver su panel principal.
- Ver sus citas.
- Agendar una nueva cita.
- Cancelar una cita pendiente.

Este controlador recibe datos desde formularios JSP, llama a los servicios correspondientes y devuelve una vista al usuario.

- **DoctorController.java:** Controla las funcionalidades del doctor.
 - [/doctor/dashboard](#)
 - [/doctor/confirmar-cita](#)
 - [/doctor/cancelar-cita](#)
 - [/doctor/paciente/{id}](#)

Desde este controlador, el doctor puede:

- Ver su agenda.
- Confirmar citas pendientes.
- Cancelar citas.
- Ver información de un paciente.

También pertenece a la capa backend, pero está directamente conectado con las vistas del frontend.

4.7. Carpeta “src/main/java/com/clinica/model ”

Esta carpeta pertenece al backend y contiene las clases que representan los datos principales del sistema. Esta carpeta contiene:

- **Usuario.java:** Representa los datos generales de una persona que puede ingresar al sistema. Contiene información como:
 - ID.
 - Nombre.
 - Email.
 - Contraseña.
 - Rol.
- **Paciente.java:** Representa a un paciente del sistema. Está compuesto por un objeto Usuario y agrega información adicional como el teléfono.
- **Doctor.java:** Representa a un doctor del sistema. Está compuesto por un objeto Usuario y agrega la especialidad médica.
- **Cita.java:** Representa una cita médica. Contiene datos como:
 - ID de la cita.
 - ID del doctor.
 - ID del paciente.
 - Fecha.
 - Hora.
 - Estado.
 - Notas.
- **EstadoCita.java:** Define los estados posibles de una cita:
 - PENDIENTE
 - CONFIRMADA
 - CANCELADA
 - COMPLETADA

4.8. Carpeta “src/main/java/com/clinica/service ”

Esta carpeta pertenece al backend y representa la capa de servicios. Aquí se definen las operaciones que el sistema puede realizar, pero de forma abstracta mediante interfaces.

Contiene:

- **UsuarioService.java:** Define operaciones relacionadas con usuarios, doctores y pacientes.
 - Buscar paciente por email.
 - Buscar doctor por email.
 - Listar doctores.
 - Buscar doctor por ID.
 - Buscar paciente por ID.
- **CitaService.java:** Define operaciones relacionadas con citas médicas.

- Buscar citas por paciente.
- Buscar citas por doctor.
- Crear cita.
- Cancelar cita.
- Confirmar cita.
- Verificar disponibilidad de horario.
- Obtener horarios disponibles.

Esta capa es importante porque separa la lógica del sistema de los controladores.

4.9. Carpeta “src/main/java/com/clinica/service/impl ”

Esta carpeta pertenece al backend y contiene la implementación real de los servicios y contiene:

- **MemoryUsuarioService.java:**
Implementa UsuarioService, su función es manejar usuarios, doctores y pacientes en memoria. Aquí se crean automáticamente los usuarios de prueba cuando inicia la aplicación. Esto significa que actualmente no se usa una base de datos para guardar usuarios.
- **MemoryCitaService.java:**
Implementa CitaService. Su función es manejar las citas médicas en memoria. Esto permite:
 - Crear citas.
 - Buscar citas.
 - Confirmar citas.
 - Cancelar citas.
 - Validar si un horario ya está ocupado.
 Esta clase contiene parte importante de la lógica de negocio del sistema.

4.10. Carpeta “src/main/java/com/clinica/repository ”

Esta carpeta pertenece al backend y está pensada para la futura capa de acceso a datos. Actualmente contiene:

- UsuarioRepository.java
- CitaRepository.java

Sin embargo, estas interfaces todavía no tienen implementación real. En este avance, los datos no se guardan en base de datos, sino en memoria. Por eso los repositorios existen como preparación para una futura conexión con MySQL usando Spring Data JPA. Esta carpeta será más importante en una siguiente etapa del proyecto.

4.11. Carpeta “src/main/resources ”

Actualmente esta carpeta contiene “application.properties”. Este archivo define configuraciones como:

- Nombre de la aplicación.
- Puerto del servidor.
- Ubicación de las vistas JSP.
- Niveles de logging.

Pertenece a la configuración general del sistema.

4.12. Carpeta “src/main/webapp ”

Esta carpeta contiene la parte visual del sistema, es decir, el frontend. Dentro de ella se encuentran los recursos estáticos y las vistas JSP.

4.13. Carpeta “src/main/webapp/resources/css”

Contiene el archivo “custom.css”, este archivo pertenece al frontend y sirve para agregar estilos personalizados a las páginas del sistema. Aunque el proyecto usa Bootstrap, este archivo

permite modificar o complementar algunos estilos.

4.14. Carpeta “src/main/webapp/WEB-INF/jsp”

Esta carpeta pertenece al frontend renderizado por el servidor. Contiene las páginas JSP que el usuario ve en el navegador.

- **login.jsp:** Pantalla de inicio de sesión, permite ingresar email y contraseña.
- **paciente/dashboard.jsp:** Panel principal del paciente, muestra sus citas y permite acceder al formulario para agendar una nueva cita.
- **paciente/agendar.jsp:** Formulario para crear una cita médica, permite seleccionar:
 - Doctor.
 - Fecha.
 - Hora.
- **paciente/mis-citas.jsp:** Pantalla donde el paciente puede revisar su historial de citas.
- **doctor/dashboard.jsp:** Panel principal del doctor, muestra la agenda de citas del doctor y permite confirmar o cancelar citas.
- **doctor/detalle-paciente.jsp:** Pantalla donde el doctor puede revisar información básica de un paciente y su historial de citas con él.
- **error.jsp:** Pantalla de error general.
- **acceso-denegado.jsp:** Pantalla que aparece cuando un usuario intenta entrar a una ruta que no corresponde a su rol.

IV. ARQUITECTURA DEL SISTEMA (UML)

V. DESARROLLO DEL SISTEMA

5.1. Clase principal de la aplicación

Archivo:

“src/main/java/com/clinica/ClinicaApplication.java”

Código:

```
@SpringBootApplication
public class ClinicaApplication {
    public static void main(String[] args) {
        SpringApplication.run(ClinicaApplication.class, args);
    }
}
```

Esta clase representa el punto de entrada principal del sistema. Al ejecutar el proyecto, Spring Boot inicia desde esta clase y carga automáticamente todos los componentes configurados dentro del paquete “com.clinica”.

La anotación “@SpringBootApplication” es fundamental porque combina varias configuraciones internas de Spring:

- Permite detectar automáticamente clases marcadas con “@Controller”, “@Service”, “@Configuration”, entre otras.
- Activa la configuración automática de Spring Boot.
- Permite iniciar el servidor embebido Tomcat.

El método “main” ejecuta la aplicación mediante:

`SpringApplication.run(ClinicaApplication.class, args);`

Esto levanta el servidor local, normalmente en:

<http://localhost:8080>

Desde ahí el usuario puede acceder al sistema mediante la ruta:

<http://localhost:8080/signin>

5.2. Configuración general del proyecto

Archivo:**“src/main/resources/application.properties”**

Código:

```
spring.application.name=clinica-citas
server.port=8080
spring.mvc.view.prefix=/WEB-INF/jsp/
spring.mvc.view.suffix=.jsp
logging.level.com.clinica=DEBUG
logging.level.org.springframework.security=INFO
```

Este archivo contiene configuraciones importantes de la aplicación.

- La propiedad:
server.port=8080
indica que el sistema se ejecuta en el puerto 8080.
- Las propiedades:
spring.mvc.view.prefix=/WEB-INF/jsp/
spring.mvc.view.suffix=.jsp
le indican a Spring MVC dónde debe buscar las vistas JSP. Por ejemplo, si un controlador retorna:
return "paciente/dashboard";
Spring buscará automáticamente el archivo:
/WEB-INF/jsp/paciente/dashboard.jsp
Esto permite que los controladores no tengan que escribir la ruta completa de cada archivo JSP.

5.3. Configuración de seguridad**Archivo:****“src/main/java/com/clinica/config/SecurityConfig.java”**

Esta clase es una de las más importantes del proyecto porque controla:

- Inicio de sesión.
- Validación de credenciales.
- Acceso según rol.
- Redirección luego del login.
- Cierre de sesión.
- Página de acceso denegado.

Código:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
```

La anotación “@Configuration” indica que esta clase contiene configuración de Spring.

La anotación “@EnableWebSecurity” activa Spring Security dentro del proyecto.

- **Validación de usuarios**

Código:

```
@Bean
public UserDetails userDetailsService() {
    return username -> {
        var pacienteOpt =
            usuarioService.findPacienteByEmail(username);
        if (pacienteOpt.isPresent()) {
            var paciente = pacienteOpt.get();
            return User.builder()
                .username(paciente.getEmail())
```

```
.password(paciente.getUsuario().getPassword())
                .roles("PACIENTE")
                .build();
        }
        var doctorOpt =
            usuarioService.findDoctorByEmail(username);
        if (doctorOpt.isPresent()) {
            var doctor = doctorOpt.get();
            return User.builder()
                .username(doctor.getEmail())
                .password(doctor.getUsuario().getPassword())
                .roles("DOCTOR")
                .build();
        }
        throw new UsernameNotFoundException("Usuario
no encontrado: " + username);
    };
}
```

Este método se encarga de buscar al usuario cuando intenta iniciar sesión.

Primero busca si el correo ingresado pertenece a un paciente:

usuarioService.findPacienteByEmail(username);

Si lo encuentra, crea un usuario de Spring Security con el rol:

.roles("PACIENTE")

Luego, si no encuentra un paciente, busca si pertenece a un doctor:

usuarioService.findDoctorByEmail(username);

Si lo encuentra, le asigna el rol:

.roles("DOCTOR")

Este rol es importante porque después Spring Security lo utiliza para permitir o bloquear el acceso a ciertas rutas.

Por ejemplo:

- Un paciente puede entrar a /paciente/**
- Un doctor puede entrar a /doctor/**.

- **Protección de rutas**

Código:

```
.authorizeHttpRequests(auth -> auth
    .dispatcherTypeMatchers(DispatcherType.FORWARD,
    DispatcherType.ERROR).permitAll()
    .requestMatchers("/signin", "/do-login",
    "/resources/**").permitAll()
    .requestMatchers("/error",
    "/acceso-denegado").permitAll()
    .requestMatchers("/paciente/**").hasRole("PACIENTE")
    .requestMatchers("/doctor/**").hasRole("DOCTOR")
    .anyRequest().authenticated()
)
```

Aquí se definen los permisos de acceso.

Las rutas:

- /signin
- /do-login
- /resources/**

son públicas. Esto significa que cualquier usuario puede acceder al login y a los archivos CSS.

Las rutas:

- /paciente/**

solo pueden ser usadas por usuarios con rol “PACIENTE”.

Las rutas:

- /doctor/**

solo pueden ser usadas por usuarios con rol “DOCTOR”. Esto ayuda a cumplir uno de los objetivos principales del

sistema: separar las funcionalidades según el perfil del usuario.

- **Configuración del login:**

Código:

```
.formLogin(form -> form
    .loginPage("/signin")
    .loginProcessingUrl("/do-login")
    .usernameParameter("username")
    .passwordParameter("password")
    .successHandler(this::authenticationSuccessHandler)
    .failureUrl("/signin?error=true")
)
```

Este bloque indica que la página de inicio de sesión será: “/signin”

El formulario enviará los datos a: “/do-login”

Los campos del formulario deben llamarse: “username” y “password”

Si el login falla, el sistema redirige a: “/signin?error=true”

Esto permite que la vista JSP muestre un mensaje de error.

- **Redirección según el rol**

Código:

```
private void authenticationSuccessHandler(
    HttpServletRequest request,
    HttpServletResponse response,
    Authentication authentication) throws IOException
{
    String redirectUrl = "/";
    if (authentication.getAuthorities().stream()
        .anyMatch(a ->
            a.getAuthority().equals("ROLE_PACIENTE"))) {
        redirectUrl = "/paciente/dashboard";
    } else if (authentication.getAuthorities().stream()
        .anyMatch(a ->
            a.getAuthority().equals("ROLE_DOCTOR"))) {
        redirectUrl = "/doctor/dashboard";
    }
    response.sendRedirect(redirectUrl);
}
```

Este método se ejecuta cuando el usuario inicia sesión correctamente.

Si el usuario tiene el rol “ROLE_PACIENTE”, será enviado a: “/paciente/dashboard”

Si tiene el rol “ROLE_DOCTOR”, será enviado a: “/doctor/dashboard”

Esto mejora la experiencia del usuario porque no necesita elegir manualmente a qué panel entrar.

5.4. Modelo de datos

En el paquete “model” se encuentran las clases que representan los datos principales del sistema.

- **Usuario:**

Archivo: “src/main/java/com/clinica/model/Usuario.java”
Esta clase contiene los datos generales de una persona que puede iniciar sesión:

- id
- nombre

- email
- password
- rol

El campo rol permite diferenciar entre doctor y paciente.

- **Paciente:**

Archivo: “src/main/java/com/clinica/model/Paciente.java”

La clase “Paciente” contiene un objeto “Usuario” y un teléfono.

Esto significa que un paciente reutiliza los datos generales de usuario, pero agrega información propia como:

```
private String telefono;
```

También tiene métodos como:

```
public Long getId()
```

```
public String getNombre()
```

```
public String getEmail()
```

Estos métodos facilitan mostrar datos del paciente en las vistas JSP.

- **Doctor:**

Archivo: “src/main/java/com/clinica/model/Doctor.java”

La clase “Doctor” también contiene un objeto “Usuario”, pero agrega:

```
private String especialidad;
```

Esto permite mostrar en el formulario de agendar cita el nombre del doctor junto con su especialidad.

- **Cita:**

Archivo: “src/main/java/com/clinica/model/Cita.java”

La clase “Cita” representa una cita médica.

```
private Long id;
private Long doctorId;
private Long pacienteId;
private LocalDate fecha;
private LocalTime hora;
private EstadoCita estado;
private String notas;
```

Explicación:

- id: identificador único de la cita.
- doctorId: indica qué doctor atenderá la cita.
- pacienteId: indica qué paciente reservó la cita.
- fecha: día de la cita.
- hora: hora seleccionada.
- estado: estado actual de la cita.
- notas: campo preparado para observaciones futuras.

La clase también contiene

```
private String nombreDoctor;
```

```
private String nombrePaciente;
```

Estos campos se usan para mostrar nombres en pantalla sin tener que buscar manualmente el doctor o paciente desde la vista JSP.

- **EstadoCita:**

Archivo: “src/main/java/com/clinica/model/EstadoCita.java”

Código:

```
public enum EstadoCita {
    PENDIENTE,
    CONFIRMADA,
    CANCELADA,
```

COMPLETADA

Este “enum” define los posibles estados de una cita médica.

- “PENDIENTE”: cuando el paciente agenda una cita.
- “CONFIRMADA”: cuando el doctor acepta la cita.
- “CANCELADA”: cuando el paciente o doctor cancela la cita.
- “COMPLETADA”: estado preparado para uso futuro.

5.5 Servicio de usuarios

Archivo:

"src/main/java/com/clinica/service/impl/MemoryUsuarioService.java"

Esta clase funciona como una base de datos temporal de usuarios.

Código:

```
private final ConcurrentHashMap<Long, Usuario> usuarios = new
ConcurrentHashMap<>();
private final ConcurrentHashMap<Long, Doctor> doctores = new
ConcurrentHashMap<>();
private final ConcurrentHashMap<Long, Paciente> pacientes =
new ConcurrentHashMap<>();
```

Explicación:

Se utilizan mapas en memoria para guardar usuarios, doctores y pacientes.

Esto significa que los datos existen solo mientras la aplicación está encendida. Si el servidor se reinicia, los datos creados durante la ejecución se pierden.

Creación automática de usuarios

Código:

```
@PostConstruct
public void initData() {
    crearDoctor("Dr. García", "dr.garcia@clinica.com", "admin123",
"Cardiología");
    crearDoctor("Dr. López", "dr.lopez@clinica.com", "admin123",
"Pediatría");
    crearPaciente("Juan Pérez", "juan@email.com", "user123",
"555-0101");
    crearPaciente("María Gómez", "maria@email.com", "user123",
"555-0102");
}
```

Explicación:

La anotación @PostConstruct hace que este método se ejecute automáticamente cuando inicia la aplicación.

Aquí se crean usuarios de prueba para poder ingresar al sistema sin necesidad de registrar usuarios desde una base de datos.

Creación de doctores

Código:

```
private void crearDoctor(String nombre, String email, String
password, String especialidad) {
    Long id = idGenerator.getAndIncrement();
    Usuario usuario = new Usuario(id, nombre, email,
passwordEncoder.encode(password), "ROLE_DOCTOR");
    usuarios.put(id, usuario);
    Doctor doctor = new Doctor(usuario, especialidad);
    doctores.put(id, doctor);
}
```

Explicación:

Este método crea un usuario con rol de doctor.

Primero genera un ID automático:

```
Long id = idGenerator.getAndIncrement();
```

Luego crea un objeto Usuario, encriptando la contraseña:

```
passwordEncoder.encode(password)
```

Después guarda el usuario en el mapa usuarios y crea un objeto Doctor con su especialidad.

Creación de pacientes

Código:

```
private void crearPaciente(String nombre, String email, String
password, String telefono) {
    Long id = idGenerator.getAndIncrement();
    Usuario usuario = new Usuario(id, nombre, email,
passwordEncoder.encode(password), "ROLE_PACIENTE");
    usuarios.put(id, usuario);
    Paciente paciente = new Paciente(usuario, telefono);
    pacientes.put(id, paciente);
}
```

Explicación:

Funciona de forma similar al doctor, pero asigna el rol:

ROLE_PACIENTE

Y además guarda el teléfono del paciente.

5.6 Servicio de citas

Archivo:

"src/main/java/com/clinica/service/impl/MemoryCitaService.java"

Esta clase contiene la lógica principal para crear, buscar, confirmar y cancelar citas.

Código:

```
private final ConcurrentHashMap<Long, Cita> citas = new
ConcurrentHashMap<>();
private final AtomicLong idGenerator = new AtomicLong(1);
```

Explicación:

Las citas se almacenan temporalmente en memoria. Cada cita recibe un ID automático generado por AtomicLong.

Crear una cita

Código:

```
public Cita crearCita(Long doctorId, Long pacienteId, LocalDate
fecha, LocalTime hora) {
    Long id = idGenerator.getAndIncrement();
    Cita cita = new Cita(id, doctorId, pacienteId, fecha, hora,
EstadoCita.PENDIENTE, null);
    citas.put(id, cita);
    return cita;
}
```

Explicación:

Cuando un paciente agenda una cita, se crea un objeto Cita con estado inicial:

EstadoCita.PENDIENTE

Esto significa que la cita todavía no ha sido confirmada por el doctor.

Después se guarda en el mapa citas.

Cancelar cita por paciente

Código:

```
public boolean cancelarCita(Long citaId, Long pacienteId) {
    Cita cita = citas.get(citaId);
    if (cita != null && cita.getPacienteId().equals(pacienteId)
        && cita.getEstado() == EstadoCita.PENDIENTE) {
        cita.setEstado(EstadoCita.CANCELADA);
        return true;
    }
    return false;
}
```

Explicación:

Este método permite que un paciente cancele una cita, pero solo si cumple estas condiciones:

La cita existe.

La cita pertenece al paciente autenticado.

La cita está en estado PENDIENTE.

Si la cita ya fue confirmada, el paciente no puede cancelarla desde esta lógica.

Confirmar cita por doctor

Código:

```
public boolean confirmarCita(Long citaId, Long doctorId) {
    Cita cita = citas.get(citaId);
    if (cita != null && cita.getDoctorId().equals(doctorId)
        && cita.getEstado() == EstadoCita.PENDIENTE) {
        cita.setEstado(EstadoCita.CONFIRMADA);
        return true;
    }
    return false;
}
```

Explicación:

Este método permite al doctor confirmar una cita.

Las condiciones son:

La cita debe existir.

La cita debe pertenecer al doctor autenticado.

La cita debe estar pendiente.

Si todo se cumple, el estado cambia a:

CONFIRMADA

Cancelar cita por doctor

Código:

```
public boolean cancelarCitaByDoctor(Long citaId, Long doctorId)
{
    Cita cita = citas.get(citaId);
    if (cita != null && cita.getDoctorId().equals(doctorId)
        && (cita.getEstado() == EstadoCita.PENDIENTE ||
        cita.getEstado() == EstadoCita.CONFIRMADA)) {
        cita.setEstado(EstadoCita.CANCELADA);
        return true;
    }
    return false;
}
```

Explicación:

El doctor puede cancelar citas que estén pendientes o confirmadas. Esto es diferente al paciente, ya que el paciente solo puede cancelar citas pendientes.

Validar disponibilidad de horario

Código:

```
public boolean isSlotDisponible(Long doctorId, LocalDate fecha,
    LocalTime hora) {
    return citas.values().stream()
        .noneMatch(c -> c.getDoctorId().equals(doctorId)
            && c.getFecha().equals(fecha)
            && c.getHora().equals(hora)
            && c.getEstado() != EstadoCita.CANCELADA);
}
```

```
        && c.getHora().equals(hora)
        && c.getEstado() != EstadoCita.CANCELADA);
}
```

Explicación:

Este método revisa si un doctor ya tiene una cita en la misma fecha y hora.

Si existe una cita con:

mismo doctor,

misma fecha,

misma hora,

y no está cancelada,

entonces el horario no está disponible.

Esto evita que dos pacientes reserven el mismo turno con el mismo doctor.

Horarios disponibles

Código:

```
public List<LocalTime> getHorariosDisponibles() {
    List<LocalTime> horarios = new ArrayList<>();
    LocalTime inicio = LocalTime.of(9, 0);
    LocalTime fin = LocalTime.of(17, 0);

    LocalTime actual = inicio;
    while (actual.isBefore(fin)) {
        horarios.add(actual);
        actual = actual.plusMinutes(30);
    }
    return horarios;
}
```

Explicación:

Este método genera los horarios disponibles desde las 9:00 hasta las 17:00, en intervalos de 30 minutos.

Ejemplo:

09:00

09:30

10:00

10:30

...

16:30

Actualmente el sistema muestra estos horarios en el formulario de agendar cita

8.7 Controlador del paciente

Archivo:

src/main/java/com/clinica/controller/PacienteController.java

Este controlador maneja todas las rutas que empiezan con: "/paciente"

• Dashboard del paciente

Código:

```
@GetMapping("/dashboard")
public String dashboard(Model model, Authentication
auth) {
    Long pacienteId = getPacienteId(auth.getName());
```

```
Paciente paciente =
usuarioService.findPacienteById(pacienteId).orElse(null);
```

```
List<Cita> citasPendientes =
citaService.findCitasByPacienteId(pacienteId);
```

```
model.addAttribute("paciente", paciente);
model.addAttribute("citas", citasPendientes);
```



```
return "paciente/dashboard";
}
```

Explicación:

Este método se ejecuta cuando el paciente entra a: “/paciente/dashboard”

Obtiene el correo del usuario autenticado mediante:

```
auth.getName()
```

Luego busca el ID del paciente y obtiene sus datos.

Después consulta sus citas y las envía a la vista JSP mediante:

```
model.addAttribute("paciente", paciente);
model.addAttribute("citas", citasPendientes);
```

Finalmente muestra la vista: “paciente/dashboard.jsp”

● **Mostrar formulario de agendar cita**

Código:

```
@GetMapping("/agendar")
public String mostrarAgendar(Model model,
Authentication auth) {
    List<Doctor> doctores =
usuarioService.findAllDoctores();
    List<LocalTime> horarios =
citaService.getHorariosDisponibles();
```

```
    model.addAttribute("doctores", doctores);
    model.addAttribute("horarios", horarios);
    return "paciente/agendar";
}
```

Explicación:

Cuando el paciente entra a: “/paciente/agendar” el sistema carga:

- lista de doctores disponibles,
- lista de horarios disponibles.

Estos datos se envían al JSP para llenar los campos del formulario.

● **Registrar nueva cita**

Código:

```
@PostMapping("/agendar")
public String agendarCita(@RequestParam Long
doctorId,
    @RequestParam String fecha,
    @RequestParam String hora,
    Authentication auth,
    RedirectAttributes redirectAttributes) {
```

Explicación:

Este método recibe los datos enviados desde el formulario:

- ID del doctor.
- Fecha.
- Hora.

Luego convierte los textos recibidos en objetos “LocalDate” y “LocalTime”.

Valida que la fecha no sea pasada:

```
if (fechaParsed.isBefore(LocalDate.now())) {
    redirectAttributes.addFlashAttribute("error", "No
puede agendar en fechas pasadas.");
    return "redirect:/paciente/agendar";
}
```

También valida que el horario esté disponible:

```
if (!citaService.isSlotDisponible(doctorId, fechaParsed,
horaParsed)) {
    redirectAttributes.addFlashAttribute("error", "El
horario seleccionado ya está ocupado.");
    return "redirect:/paciente/agendar";
}
```

Si todo es correcto, crea la cita:

```
citaService.crearCita(doctorId, pacienteId, fechaParsed,
horaParsed);
```

y redirige al dashboard.

5.8 Controlador del doctor

Archivo:

src/main/java/com/clinica/controller/DoctorController.java

Este controlador maneja las rutas que empiezan con: “/doctor”

● **Dashboard del doctor**

Código:

```
@GetMapping("/dashboard")
public String dashboard(Model model, Authentication
auth) {
    Long doctorId = getDoctorId(auth.getName());
```

```
    Doctor doctor =
usuarioService.findDoctorById(doctorId).orElse(null);
    List<Cita> citas =
citaService.findCitasByDoctorId(doctorId);
```

```
    model.addAttribute("doctor", doctor);
    model.addAttribute("citas", citas);
    return "doctor/dashboard";
}
```

Explicación:

Este método muestra la agenda del doctor autenticado.

Obtiene el correo del doctor desde Spring Security, busca su ID, carga sus datos y luego consulta todas las citas asociadas a ese doctor.

La información se envía a: “doctor/dashboard.jsp”

● **Confirmar cita**

Código:

```
@PostMapping("/confirmar-cita")
public String confirmarCita(@RequestParam Long
citaId,
    Authentication auth,
    RedirectAttributes redirectAttributes) {
    Long doctorId = getDoctorId(auth.getName());
```

```
    boolean resultado = citaService.confirmarCita(citaId,
doctorId);
```

Explicación:

Este método permite que el doctor confirme una cita pendiente.

Recibe el citaId desde el formulario del dashboard y llama al servicio:

```
citaService.confirmarCita(citaId, doctorId);
```

Si el resultado es correcto, muestra un mensaje de éxito.

● **Cancelar cita**

Código:

```
@PostMapping("/cancelar-cita")
public String cancelarCita(@RequestParam Long citaId,
    Authentication auth,
    RedirectAttributes redirectAttributes) {
    Long doctorId = getDoctorId(auth.getName());
```

```
    boolean resultado =
citaService.cancelarCitaByDoctor(citaId, doctorId);
```

Explicación:

Este método permite que el doctor cancele una cita pendiente o confirmada.

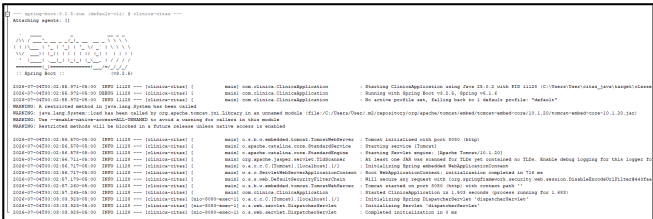
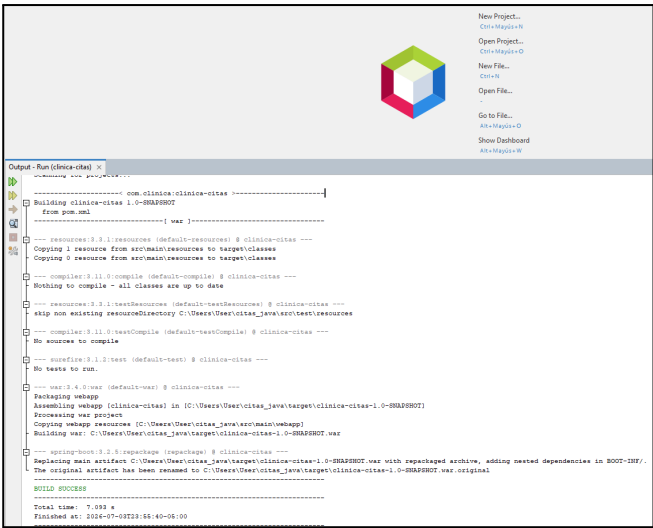
La lógica valida que la cita pertenezca al doctor

autenticado antes de cambiar su estado.

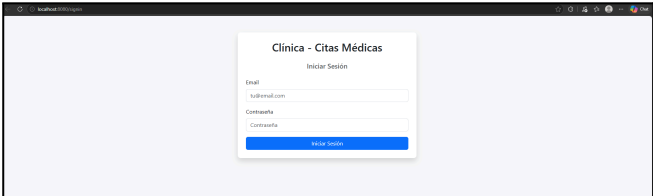
- **Ver detalle del paciente**

Código:
`@GetMapping("/paciente/{id}")`
`public String detallePaciente(@PathVariable Long id,`
`Model model, Authentication auth) {`
Explicación:
Esta ruta permite al doctor ver información de un paciente específico.
El sistema busca al paciente por su ID y luego carga sus citas. Después filtra para mostrar solo las citas que ese paciente tuvo con el doctor autenticado.
Esto evita que un doctor vea citas del paciente con otros doctores.

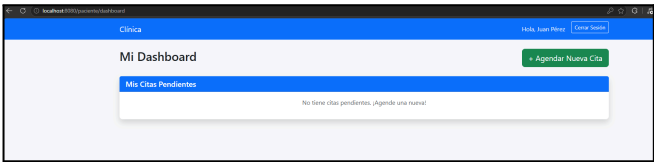
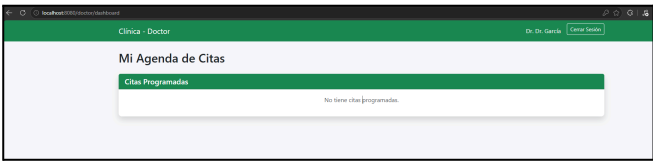
VI. ESTADO ACTUAL DEL PROYECTO



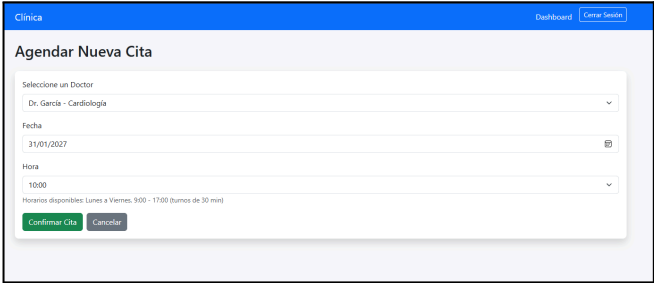
El sistema compila correctamente, tenga en cuenta al ser un proyecto en Spring boot se corre con Run Maven.



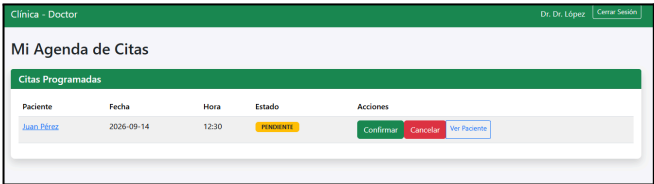
Interfaz del sistema



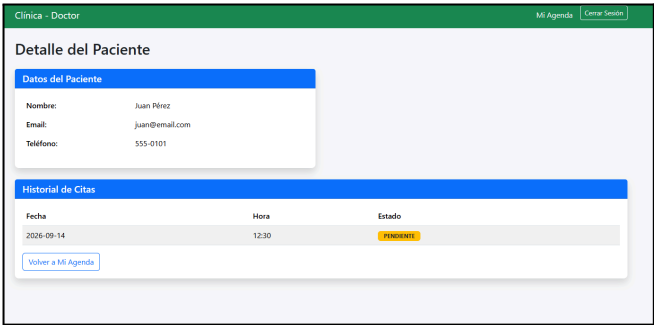
Funcionamiento correcto de ambos login.



El paciente puede agendar citas



El doctor puede confirmar y cancelar las citas



Los datos se guardan temporalmente en la memoria. No existe todavía una base de datos real.

VII. OBSERVACIONES Y SIGUIENTES AVANCES

- El backend actual es funcional, pero temporal.
- Las citas se pierden al reiniciar la aplicación.
- Los repositorios están preparados para futura implementación con MySQL.
- Falta validar que solo se pueda agendar de lunes a viernes.
- Después de realizar un correcto backend, realizar una cadena de conexión del servidor.

VIII. REFERENCIAS

[1] C. S. Horstmann, Core Java Volume I—Fundamentals, 13th ed. Hoboken, NJ, USA: Pearson, 2025.

[2] Oracle Corporation, “The Java™ Tutorials and Java Documentation,” Oracle. Available: <https://docs.oracle.com/en/java/>

[3] Object Management Group (OMG), “Unified Modeling Language (UML) Specification.” Available:
<https://www.omg.org/spec/UML/>

[4] PlantUML, “PlantUML Official Documentation.” Available:
<https://plantuml.com/>

[5] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed. Boston, MA, USA: Addison-Wesley, 2004.